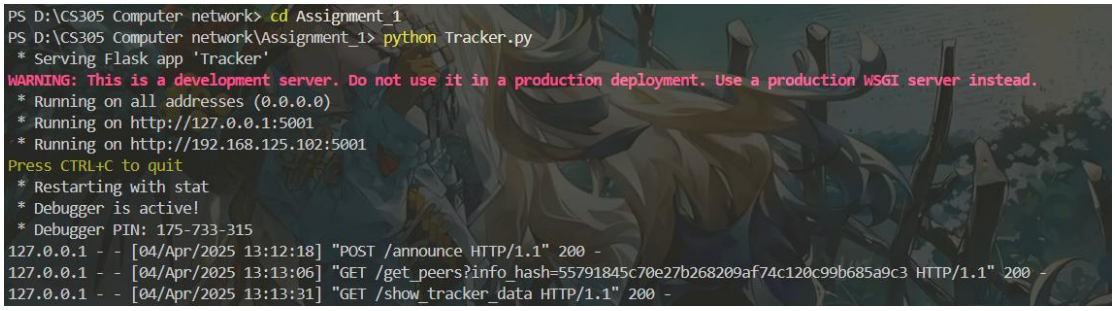
Computer Network Assignment 1　report

1

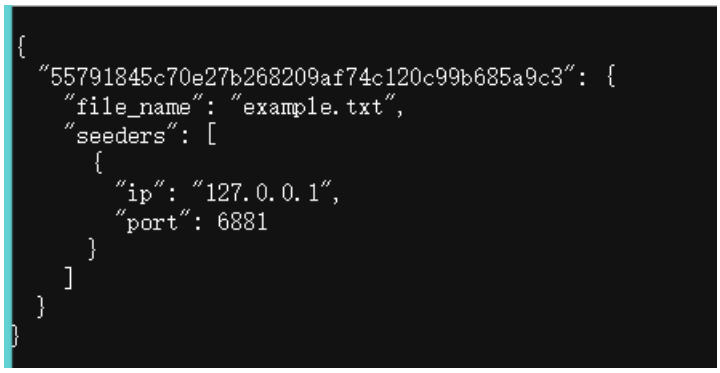
This is a assignment report about the first programming assignment of the computer networking. For this report, I will show the picture of the executing progress of three codes, which is tracker, seeder and requester, and I will explain the result. You need to know that the executing sequence is tracker the first, then the seeder and finally the requester. You also need to know that the example.txt need to be allocated in the same page with these codes.

2.1 Tracker

The result of executing the tracker in the terminal is shown below.



and the website called 127.0.0.1:5001/show_tracker_data can show the picture below



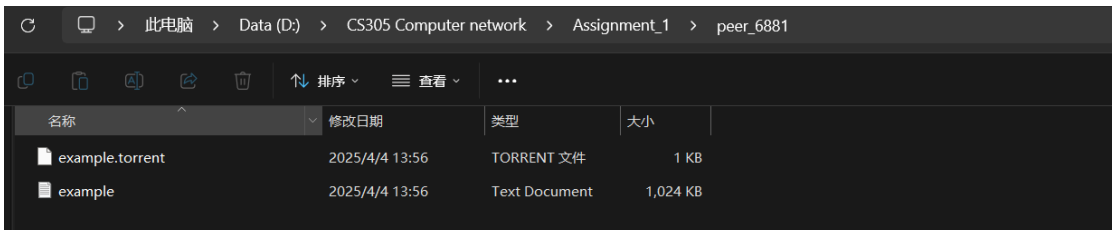
The explanation for the result of what the terminal and website show is that the following:

After running Tracker.py, the server starts and listens on port 5001. When the Seeder registers and the Requester requests the seed list, the terminal displays the corresponding HTTP request logs. Accessing the /show_tracker_data route confirms that the Tracker correctly records the info_hash and seed node information.

2.2 Seeder

The result of executing the seeder in the terminal is shown below.

And the profile of peer_6881 is shown like this, which contain the torrent and txt file.



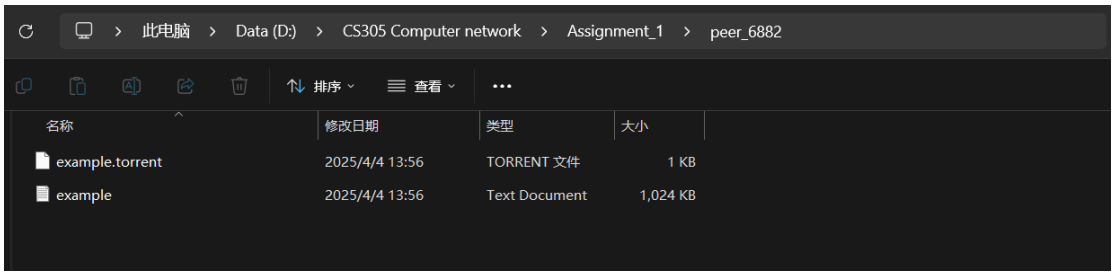
The explanation for the result of what the terminal and website show is that the following:

After running Seeder.py, the peer_6881 folder is created, example.txt is copied to this folder, and the example.torrent file is generated. The terminal displays a successful registration with the Tracker, and a screenshot of the folder confirms the file generation.

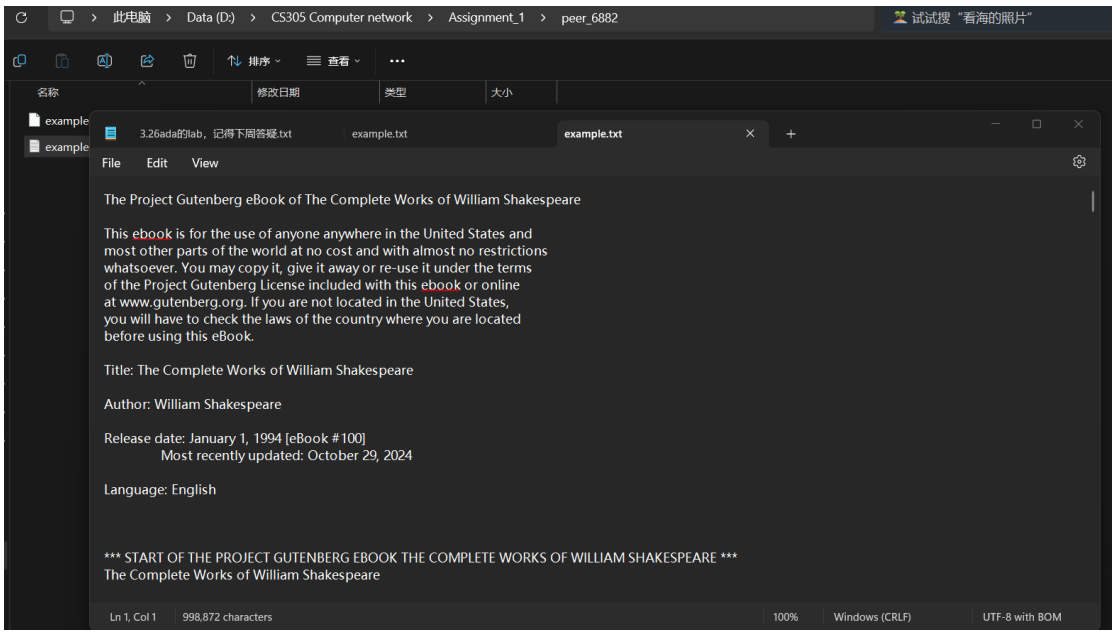
2.3 Requester

The result of executing the requester in the terminal is shown below.

And the profile of peer_6881 is shown like this, which contain the torrent and txt file.



And you can see the following picture of the example.txt .



The explanation for the result of what the terminal and file show is that the following:

After running Requester.py, the peer_6882 folder is created, the info_hash is decoded from example.torrent, and the seed node list is successfully obtained from the Tracker. The terminal displays that the file download was successful, and a screenshot of the folder along with the file contents confirms that example.txt has been downloaded.

3 Conclusion

This project successfully implements a simplified BitTorrent-based peer-to-peer (P2P) file-sharing system, consisting of three main components: Tracker.py, Seeder.py, and Requester.py. The system adheres to the assignment requirements and demonstrates the core principles of P2P file distribution using HTTP protocols.

Tracker Implementation: Tracker.py serves as the coordinator, running on port 5001. It listens for registration requests from the Seeder via the /announce endpoint and provides a list of seeders to the Requester through the /get_peers endpoint. The /show_tracker_data route confirms that the tracker accurately records the info_hash, file name, and seeder details (e.g., IP and port), fulfilling Step 1 of the testing process.
Seeder Implementation: Seeder.py, operating on port 6881, creates a .torrent file for example.txt using the bencodepy library and registers it with the tracker. The file is copied (not moved) to the peer_6881 folder to preserve the local copy, aligning with P2P principles. The /download endpoint enables file sharing, completing Step 2 successfully.
Requester Implementation: Requester.py, running on port 6882, decodes the .torrent file to obtain the info_hash, requests the seeder list from the tracker, and downloads example.txt into the peer_6882 folder. The process, validated by terminal output and file content, meets the requirements of Step 3.
The system was tested by executing the scripts in sequence: starting with Tracker.py, followed by Seeder.py, and finally Requester.py. Terminal outputs, folder contents, and the downloaded file confirm that the file-sharing process works as intended. A notable modification was made to Seeder.py, replacing shutil.move() with shutil.copy() to retain the original example.txt locally, enhancing the P2P model's realism.
In conclusion, this project effectively demonstrates a functional BitTorrent-like P2P system, achieving seamless file sharing between peers while meeting all specified grading criteria. The implementation provides a solid foundation for understanding distributed file-sharing mechanisms.

4 Appendix

The final profile structure is like this.